<FITRA>

CHANGE MANAGEMENT PLAN

Version <1.1>
<04/11/2026>

VERSION HISTORY

Version #	Implemented By	Revision Date	Approved By	Approval Date	Reason
0.1	*Mathew Wong*	15/10/2026	*Gary*	15/10/2026	*Initial template creation*
0.2	*Mathew Wong*	21/10/2026	*Gary*	21/10/2026	*Modifications to the contents*
0.3	*Mathew Wong*	25/10/2026	*Gary*	25/10/2026	*Modifications to the contents*
1.0	*Mathew Wong*	28/10/2026	*Gary*	28/10/2026	*Baseline version*
1.1	*Mathew Wong*	04/11/2026	*Gary*	04/11/2026	*corrected the historical revision and approval dates to the October project sequence.*

UP Template Version: 30/11/06

TABLE OF CONTENTS

1 INTRODUCTION **3**
1.1 PURPOSE OF THE CHANGE MANAGEMENT PLAN 4
2 CHANGE MANAGEMENT PROCESS **4**
2.1 CHANGE REQUEST PROCESS FLOW REQUIREMENTS 4
2.2 CHANGE REQUEST FORM AND CHANGE MANAGEMENT LOG 5
2.3 EVALUATING AND AUTHORIZING CHANGE REQUESTS 6
2.3.1 Change Control Board 8
3 RESPONSIBILITIES **8**
Appendix B: References **9**
Appendix C: Key Terms **10**

1 INTRODUCTION

1.1 PURPOSE OF THE CHANGE MANAGEMENT PLAN

This plan controls changes to FITRA's agreed requirements, software, database, interfaces, tests and project documents. It provides a traceable route from a reported problem or proposed improvement to an approved, tested and released change.

FITRA is the SC3040 fitness and health tracking web application. Its scope covers account access and profiles, friends, the Home dashboard, Strava connection, meal analysis and logging, training planning and readiness, daily check-ins, leaderboards and shared challenges.

The plan applies to the Next.js/TypeScript frontend, FastAPI/Python backend, Supabase authentication and database, Gemini integration, readiness model artifacts and third-party dependencies. Jira holds change records, GitHub holds code and pull requests, and Google Drive holds controlled project documents. The intended readers are the six project members, the tutor and project evaluators. The procedures below are proposed for team approval and complement the Configuration Management Plan and Release Plan.

2 CHANGE MANAGEMENT PROCESS

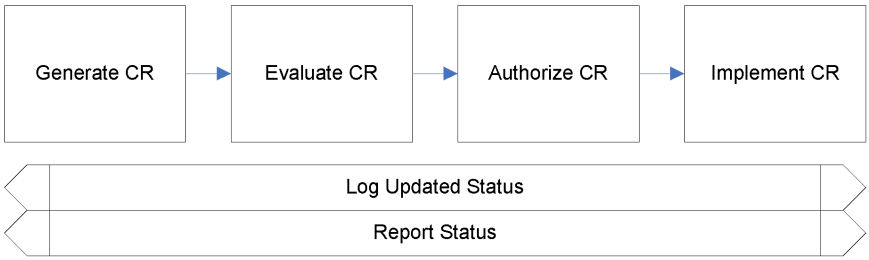
The Change Management process establishes an orderly and effective procedure for tracking the submission, coordination, review, evaluation, categorization, and approval for release of all changes to the project's baselines.

2.1 CHANGE REQUEST PROCESS FLOW REQUIREMENTS

The standard route is Open → Evaluation → Approved → Work in Progress → In Review → Testing → Ready for Release → Closed. Rejected and Deferred are recorded decisions with reasons. A failed review or test returns the change to Work in Progress; a failed release is reopened and linked to its corrective action.

Step	Description
Submit and register	Any member creates a Jira issue using Appendix D. The Change Manager checks completeness, identifies the affected requirement and baseline, and records the canonical CR identifier as FITRA-CR-<Jira issue key>.
Assess impact	The feature owner and Lead Developer identify affected modules, API consumers, database objects, AI data paths and documents. Estimate development, testing and migration effort, free-tier usage, schedule impact, risks and rollback. QA defines acceptance tests.
Decide and schedule	The decision authority in Section 2.3 approves, rejects, defers or requests clarification. Record names, date, rationale, conditions and target release in Jira. Approval is required before changing a controlled baseline.
Implement and verify	The assigned developer creates codex/<issue-key>-<short-description>, makes a focused change and links the pull request to Jira. An independent reviewer checks the diff; QA records the relevant tests. Failed checks return to the developer.

Release and close	The Release Manager verifies the approved commit, migration order, configuration and rollback steps. The Project Manager authorises delivery. After deployment and smoke tests, update the release record and baseline; close the CR only when evidence and stakeholder notification are recorded.



2.2 CHANGE REQUEST FORM AND CHANGE MANAGEMENT LOG

The Jira issue is the authoritative change request and log entry. The Google Drive log in Appendix E is a review snapshot linked back to Jira, rather than a second independent status record. Do not put passwords, tokens, meal photos or identifiable health records into tickets, pull requests or change logs.

Element	Description
Dates	Submission, decision, planned implementation, actual release and closure dates.
CR identifier	FITRA-CR-<Jira issue key>; preserve the original Jira issue key and all linked records.
Title and type	A concise behaviour change; classify as corrective, adaptive, perfective or preventive maintenance, with the impact categories in Section 2.3.
Problem and proposal	Current versus requested behaviour, reason, alternatives and measurable acceptance criteria. Link the affected SRS requirement or use case.
Submitter and owner	Requesting member, assigned feature owner and reviewers; use project identities already available in Jira.
Impact assessment	Affected components, interfaces, schema/data, privacy, effort, dependencies, cost/quota, schedule and documentation.
Evidence and decision	Reproduction steps, review comments, approval or rejection rationale, decision makers and decision date.
Product and baseline	FITRA module, current commit/tag and relevant document revision; identify each item being changed.
Delivery and rollback	Target release, pull request, commit, migration IDs, verification evidence, fallback steps and actual released version.
Priority and status	Critical/High/Medium/Low, business reason, current workflow status, next action and responsible member.

2.3 EVALUATING AND AUTHORIZING CHANGE REQUESTS

Assess priority by impact and urgency, not by the size of the edit. A small authentication, ownership, nutrition calculation or readiness-rule change can be high risk. The assessment must show which accepted requirements are preserved or deliberately revised.

Change requests are evaluated using the following priority criteria:

Priority	Description
Critical	Critical: data exposure or loss, unusable authentication, widespread outage or a release-blocking core failure. Notify the Project Manager, Lead Developer and QA immediately; use the emergency route.
High	High: a required workflow is incorrect or unavailable without a reasonable workaround. Assess promptly and prioritise within the current release if capacity and risk permit.
Medium	Medium: a contained defect or improvement with a workaround. Schedule through the backlog. Database changes require technical and QA review regardless of priority.

Low	Low: wording, documentation or cosmetic changes with no behavioural impact. Record and review them without displacing required release work.

Change requests are evaluated and assigned one or more of the following change types:

Type	Description
Scope	Add, remove or revise an agreed requirement or use case; obtain CCB approval and revise the SRS/backlog.
Schedule	Move a sprint commitment, freeze date or release milestone; record the impact on dependent work.
Effort	Change the estimated person-hours or task duration, including testing, documentation and integration.
Cost and quota	Change hosting, storage or external API use. The project's stated $0 budget requires explicit approval of any paid commitment.
Resources	Reassign responsibility or introduce a dependency on another member's availability.
Deliverables	Revise required lab documents, test evidence, coverage reports or demonstration content.
Product and interfaces	Change frontend behaviour, API contracts, calculations, model inputs/outputs or third-party integration.
Process	Change branching, review, change approval, testing or release procedures.
Quality and privacy	Change acceptance criteria, access control, data handling, reliability or test obligations.
Dependencies and database	Upgrade a library/model/service configuration or change schema/RLS/RPC behaviour. Record exact versions and migration effects.

Change requests are evaluated and assigned one of the following status types:

Status	Description
Open / Evaluation	Submitted; the Change Manager validates the record and coordinates impact assessment. No baseline change is yet authorised.
Approved / Work in Progress	Approved: decision and release recorded. Work in Progress: an assigned developer is implementing the accepted scope.
In Review	Implementation is complete, and the linked pull request is under independent review. Record unresolved comments.
Testing / Ready for Release	Testing: QA executes acceptance and regression checks. Ready for Release: required evidence passes, and release prerequisites are satisfied.
Closed	Released, smoke-tested and documented; requester or feature owner confirms acceptance. The log links to the released tag and evidence.

Rejected / Deferred	Rejected: not accepted, with rationale. Deferred: retained for later consideration with a review trigger or target date. Neither permits implementation against a controlled baseline.

2.3.1 Change Control Board

The proposed CCB consists of the Project Manager, Lead Developer and QA Manager below. Rusyaidi records decisions and advises on configuration; Quah Yew Yi advises on release readiness; the affected feature owner attends when needed. Ordinary contained changes within approved scope require the Project Manager's approval after technical and QA assessment. Scope, architecture, shared-schema, AI/privacy, cost, milestone and freeze-exception changes require explicit agreement from all three CCB members. Disagreement leaves the CR unapproved. Request course-stakeholder direction where an assessed deliverable would change. These authority rules take effect only after team approval of this plan.

Role	Name	Contact	Description
Project Manager / Chair	Gary Quah	Jira / project channel	Owns scope and schedule; records the final delivery authorisation.
Lead Developer	Matthew Wong	Jira / project channel	Assesses design, interfaces, data migration and technical feasibility.
QA Manager	Lau Jing Jie	Jira / project channel	Defines verification requirements and accepts the test evidence.

3 RESPONSIBILITIES

All members may submit and implement changes within their assigned features. Lee Zheng Wee and Rusyaidi support test design, execution and documentation under Lau Jing Jie. Reviewers must be independent of the change author; where one member holds several roles, another member performs the review. Rusyaidi serves as Change Manager and SCM, coordinating configuration records with Quah Yew Yi as Release Manager.

Role	Name	Contact	Description
Project Manager	Gary Quah	Jira / project channel	Approves scope, priority and release decisions; resolves schedule conflicts and informs affected stakeholders.
Change Manager / SCM	Rusyaidi	Jira / project channel	Registers and tracks CRs, records CCB decisions, maintains baseline/release links and coordinates migrations and notifications.
Development and QA	Matthew Wong; Lau Jing Jie; Lee Zheng Wee; Rusyaidi; affected feature owners	Jira / pull requests	Lead Developer assesses design; developers implement; QA Manager and engineers verify acceptance, regression and privacy controls.

Appendix A: Change Management Plan Approval

The named reviewers below should confirm the workflow, CCB authority and Rusyaidi's Change Manager/SCM responsibilities. Once approved, issue revision V1 and record subsequent changes through the same process.

Review record: Gary Quah — scope, schedule and authority; Matthew Wong — technical controls and integration; Lau Jing Jie — quality gates and evidence. Resolve review comments before baselining.

Signature:	GQ	Date:	21/10/2026
Print Name:	Gary Quah		
Title:	Project Manager		
Role:	Plan approver		

Signature:	MW	Date:	21/10/2026
Print Name:	Matthew Wong		
Title:	Lead Developer		
Role:	Technical reviewer		

Signature:	JJ	Date:	21/10/2026
Print Name:	Lau Jing Jie		
Title:	QA Manager		
Role:	Quality reviewer		

Appendix B: References

Use the latest team-approved revision of each reference. The source document contains drafts and historical copies; this plan does not treat every tab as an approved baseline.

Project references are held in the supplied SC3040 Google Doc and the working repository. The specific reference set is listed below.

Document Name and Version	Description	Location
SC3040 project documentation Latest approved revisions	Project Plan; SRS Edit; Quality Plan; Risk Management; Lab04 Configuration Management, Maintainability and Release Plan. Use these for requirements, roles, milestones, baselines and quality criteria.	Source SC3040 Google Doc

Appendix C: Key Terms

Repository references: README.md; AGENTS.md; docs/meal-analysis-setup.md; docs/nutrition-calendar.md; backend/app/training/README.md; backend/app/training/ml/README.md; versioned Supabase migrations and the relevant automated tests. Capture the reviewed commit when approving a CR; live service state is not established by repository documentation alone.

Interpretation rule: implementation evidence describes the inspected code, while approved requirements define the intended baseline. Record a CR for any discrepancy that needs a requirement, code or documentation change.

Key terms used in the workflow:

Term	Definition
Change request (CR)	A recorded proposal to correct or revise an accepted requirement, implementation, configuration or document.
Baseline/configuration item	A baseline is an approved reference set. A configuration item is a controlled component such as frontend code, backend code, database migration, model artifact or document.
CCB / SCM / SOUP	CCB: Change Control Board. SCM: software configuration management role. SOUP: third-party software/dependencies, using the Configuration Management Plan’s terminology.

Appendix D: Change Request Form Example

The example Change Request Form attached below can be used to submit changes during the life of the project.



CDC_UP_Change_Request_Form_Example

Appendix E: Change Management Log Template

The detailed Change Management Log template attached below can be used to track and manage requested changes during the life of the project.



CDC_UP_Change_M
anagement_Log_Tem